

Uffizi RL — Reseed Plan: My Understanding

*A faithful, plain-language read of docs/reseed_plan_updated.pdf (Phases A–G, dated 2026-06-17) and the coordinator/many-core section of docs/reseed_teammate_prompt.md. This document records what the plan asks for and **why**, before any heavy compute is spent.*

1. The problem in one paragraph

Every model in the Uffizi RL project was trained **once**, under a single training seed (42). A single seed is a single stochastic draw, so the headline numbers (the +32–41% booking lift, the “book early, earlier when busier” lesson, “MaskablePPO wins”) rest on **one lucky run with no error bars**. The reseed task fixes this: we add **four more independent training seeds** and average all five, turning every reported number into a **mean $\pm$ standard deviation over $n = 5$ seeds**.

2. The single thing that changes — and everything that does not

Changes	Only the training seed , passed as <code>--seed</code> .
Stays frozen	Environment, rewards, room capacities, the crowd simulator, every hyperparameter, the training budgets, and the six fixed CRN evaluation crowd seeds 900000..900005.

This is the whole statistical argument. Because only the training seed moves while every other knob is held fixed, the runs are **four independent stochastic replications of the same experiment**. The evaluation crowd seeds are *not* training seeds; they are held fixed on purpose so that score gaps between algorithms reflect the **learned policy**, not luck in which crowd was sampled (common-random-number variance reduction).

The seed set

```
new training seeds: 1   3   7   202606
committed seed:    42
                    -----
total:             n = 5
```

Seed 42 is already committed (its two result files already sit in `outputs/seeds/`), so we only need to **produce the four new seeds**. We must never use 42 as a new seed, and never overwrite its files.

3. What gets produced for each of the four new seeds

Two independent models are trained per seed:

1. **Toy tabular Q-learning** (the sanity model) — `uffizi_rl/pipeline/02_train_q_learning.py --seed <s> --episodes 25000`. Fast (minutes). Trains vanilla Q-learning and Double-Q on the 12-room toy graph. Its output `outputs/02_q_learning.json` is **copied** to a seed-stamped name `outputs/seeds/q_learning_seed<s>.json`.
2. **The 30-cell booking matrix** (the workhorse) — `scripts/reseed_booking_grid.py --seed <s>`. Heavy: $\sim 3.6\text{M}$ training steps per seed. Produces `outputs/seeds/results_rl_booking_seed<s>.json`.

The 30-cell matrix = 5 algorithms $\times$ 2 profiles $\times$ 3 crowds

- **Algorithms (arms):**
 - `ppo_base` — PPO walk, no interventions (baseline, 150k steps, evaluated plain)

- **dqn_base** — DQN walk, no interventions (baseline, 150k steps, evaluated plain)
- **mppo_int** — **MaskablePPO RAMA booking — the centerpiece** (100k steps, evaluated *masked*)
- **ppo_int** — unmasked PPO RAMA booking (100k steps, evaluated plain)
- **dqn_int** — DQN RAMA booking (100k steps, evaluated plain)
- **Profiles:** **art** (art lover), **tourist** (normal tourist).
- **Crowds:** 500, 2500, 5000 daily visitors.
- **Headline grid:** **mppo_int** (intervened) vs **ppo_base** (baseline). The % vs base lift is recomputed from the per-seed means.

Each policy is evaluated **deterministically** on the same six fixed CRN crowds. Models are saved **seed-stamped** as `{arm}_{profile}_{crowd}_seed{seed}.zip` under `outputs/models/seeds/`, so seeds never collide with one another or with the seed-42 models in `outputs/models/newenv/`. The driver is **per-cell resumable**: if a seed-stamped zip already exists it is reused, so an interrupted run can simply be rerun.

Compute budget: $\sim 3.6\text{M steps} \times 4 \text{ seeds} \approx 14.4\text{M training steps}$. On this 32-core machine with a 12-wide worker pool (each worker capped to 2 CPU threads), the heavy phase takes **several hours**.

4. Why parallel training is safe — the one subtlety

We have $4 \text{ seeds} \times 30 \text{ cells} = 120 \text{ independent seed-cell training jobs}$. They can run in parallel because each job is keyed by a unique (**seed**, **arm**, **profile**, **crowd**) tuple and writes a **unique** model zip — process scheduling is not part of the experiment.

The **one subtlety** the plan designs around: the booking driver rewrites a seed’s *results JSON* using only the **subset of cells** requested in that invocation. So when many same-seed jobs run in parallel, the per-seed JSON produced during the parallel phase is **partial / non-authoritative**.

The resolution is a clean split of responsibilities:

- **Phase B (parallel) produces the durable artifact = the 120 cached model zips.** Its JSONs are throwaway.
- **Phase C (sequential, one writer per seed) produces the authoritative JSON.** It reloads the cached models for all 30 cells and writes the complete file in a single pass.

5. The phases, A $\rightarrow$ H

Phase	What it does	Key guard
A	<code>uv sync</code> ; optional preflight <code>pytest</code> (deterministic/reproducible tests); train toy Q for each seed and copy to seed-stamped name.	Locked deps; fast deliverable first.
A2	Sequential-vs-parallel smoke check (gate). Build two isolated repo copies (<code>repo_seq</code> , <code>repo_par</code> , excluding <code>outputs/</code>), run 4 sampled cells (1 <code>ppo_base art</code> 500, 7 <code>dqn_base tourist</code> 2500, 3 <code>mppo_int art</code> 5000, 202606 <code>ppo_int tourist</code> 500) sequentially and through the parallel path (the <code>dqn_base</code> cell now uses seed 7), and require the canonical JSON outputs to match exactly .	Do not start Phase B until this passes.

Phase	What it does	Key guard
B	Train all 120 booking cells in parallel (<code>xargs -P 12</code>). Log everything; check exit status and <code>grep</code> for errors.	Partial JSONs here are not results; the cached zips are.
B2	Verify cached-model coverage — every expected <code>{arm}_{profile}_{crowd}_seed{s}.zip</code> is present (120 of them).	Use exact filenames, not loose globs.
C	Assemble each seed’s authoritative JSON — one sequential full-matrix pass per seed; reloads cached models (no retraining) and writes the complete 30-cell file with a single writer.	Never two assemblies for the same seed at once.
D	Sanity-check every result file — valid JSON, no NaN/Inf, 6 <code>profile×crowd</code> cells × 5 arms, and every <code>mppo_int</code> check-in count in <code>[0, 3]</code> .	A check outside <code>[0,3]</code> means the run failed.
E	Average all five seeds — <code>scripts/average_seed_runs.py</code> prints the booking grid and the toy grid with mean ± std and an <code>n</code> column.	Expect n = 5 .
F	Hand off the 8 small files in <code>outputs/seeds/</code> (4 booking + 4 toy).	Never commit/hand back the large model zips; do not commit unless asked.
G	Write the implementation report <code>docs/reseed_implementation_report.pdf</code> — what was run, which files were created, which logs were inspected, and that each seed’s final JSON came from the Phase C assembly pass.	Plain-spoken, precise.
H	Reproducibility — the last step. <code>python run_project.py --reseed</code> reproduces the whole reseed from scratch via the portable <code>scripts/reseed_reproduce.py</code> ; the new <code>scripts/docs/result</code> files are added to the repo so a fresh clone regenerates everything.	CPU-forced; per-cell resumable.

6. Guardrails (the “do not touch” list)

- **Never** pass `--eval-existing` — it scores the committed seed-42 models and would overwrite `results_rl_booking_seed42.json`.
- **Never** run pipeline scripts 07 / 08 / 09 directly — they reuse the seed-42 models and would retrain nothing.

- **Never** alter the environment, rewards, capacities, simulator, hyperparameters, evaluation seeds, or budgets. **Only --seed moves.**
- **Never** treat a Phase-B partial JSON as a final result — the authoritative JSON is always the Phase C single-writer pass.
- Keep Phase C **single-writer per seed**; do not run two assemblies for the same seed concurrently.
- **Model zips are not deliverables** — `outputs/models/seeds/` and the throwaway `outputs/checks/` are now in `.gitignore`, so they stay out of commits; only the small result JSONs and the scripts are committed.
- **On any unexpected error: STOP and report it verbatim.** Do not work around it.

7. Definition of done

- ☐ Phase A2 canonical-JSON smoke check passes for all four sampled seed-cells.
- ☐ Four `results_rl_booking_seed{1,3,7,202606}.json` — valid JSON, no NaN/Inf, `mppo_int` checks $\in [0, 3]$.
- ☐ Four `q_learning_seed{1,3,7,202606}.json` — valid JSON, no NaN/Inf.
- ☐ `scripts/average_seed_runs.py` prints booking + toy grids with `n = 5`.
- ☐ This understanding document and the final A–F implementation report are both written and compiled to PDF.
- ☐ No model zips committed or handed off.
- ☐ **Reproducible from scratch** — `python run_project.py --reseed` regenerates everything; the new scripts and result files are added to the repo (Step H, the last step).

8. Facts I verified before starting (so execution is predictable)

- Tooling present: `git 2.43.0`, `uv 0.9.22`, `pdflatex / xelatex / latexmk / pandoc`; **32 CPU cores**; `config.DEFAULT_SEED = 42`.
- All **30 seed-42 models** exist in `outputs/models/newenv/`; `outputs/models/seeds/` is **empty** (so all 120 new cells train from scratch). Seed-42's two result files already live in `outputs/seeds/`, so the averager reaches `n = 5` without `--include-canonical`.
- The driver self-roots (`sys.path.insert + os.chdir(ROOT)`) and caps itself to **2 CPU threads**.
- The optional Phase-A preflight filter `-k "deterministic or reproducible"` selects **three real tests**: `test_type_b_deterministic`, `test_reproducible`, `test_reset_deterministic`.
- A2 is a clean reproducibility gate because each sampled cell uses a **distinct seed**, so each `results_rl_booking_seed<s>.json` has exactly **one writer** in both the sequential and parallel runs — only the scheduling differs, which is exactly what the check isolates.

9. Reproducibility — Step H (the last step)

The final step makes the whole experiment reproducible from a fresh clone:

- **One entry point:** `python run_project.py --reseed` reproduces everything from scratch. It delegates to the portable `scripts/reseed_reproduce.py`, which runs Phase A (toy Q per seed) → Phase B (the 120 booking cells in a CPU process pool) → Phase C (single-writer assembly per seed) → Phase D (sanity) → Phase E (grand average, `n = 5`). Useful flags: `--seeds`, `--workers`, `--skip-train` (assemble/average from cached models only).
- **Portable & deterministic:** every script derives the repo root from its own location (no hardcoded paths), and training is pinned to CPU via `CUDA_VISIBLE_DEVICES=""`, so results are bit-reproducible (the A2 gate confirmed this) and a re-run reuses cached model zips.
- **Added to the repo:** the new helper scripts (`scripts/reseed_reproduce.py`, `scripts/phase*_{sh,py}`), the `run_project.py --reseed` wiring, the two docs, and the 8 small result JSONs in `outputs/seeds/` are committed. The large model zips (`outputs/models/seeds/`) and the throwaway smoke-check workspace (`outputs/checks/`) are `.gitignored` — they regenerate on demand.